

---

# **Event Stream Processing**

Implementation based on Apache Kafka (Step by Step Guide)

Marc Schaaf

## 1 Case Implementation

For the implementation we simplify the scenario a bit to only track if our drivers are delayed while on route to the customer. Thus we will not include any job/task related data.

- **Data available:**

- Frequent GPS based reports of the drivers while they are on route to a job.
- Planned arrival times and routes

- **Goal:**

- Determine if a driver is delayed
- Determine if the delay is significant enough to notify the customer
- Have a dashboard which shows delayed drivers so that the customer service can respond to them.

\* A *very simple* dashboard is already provided

- **Sample Code:**

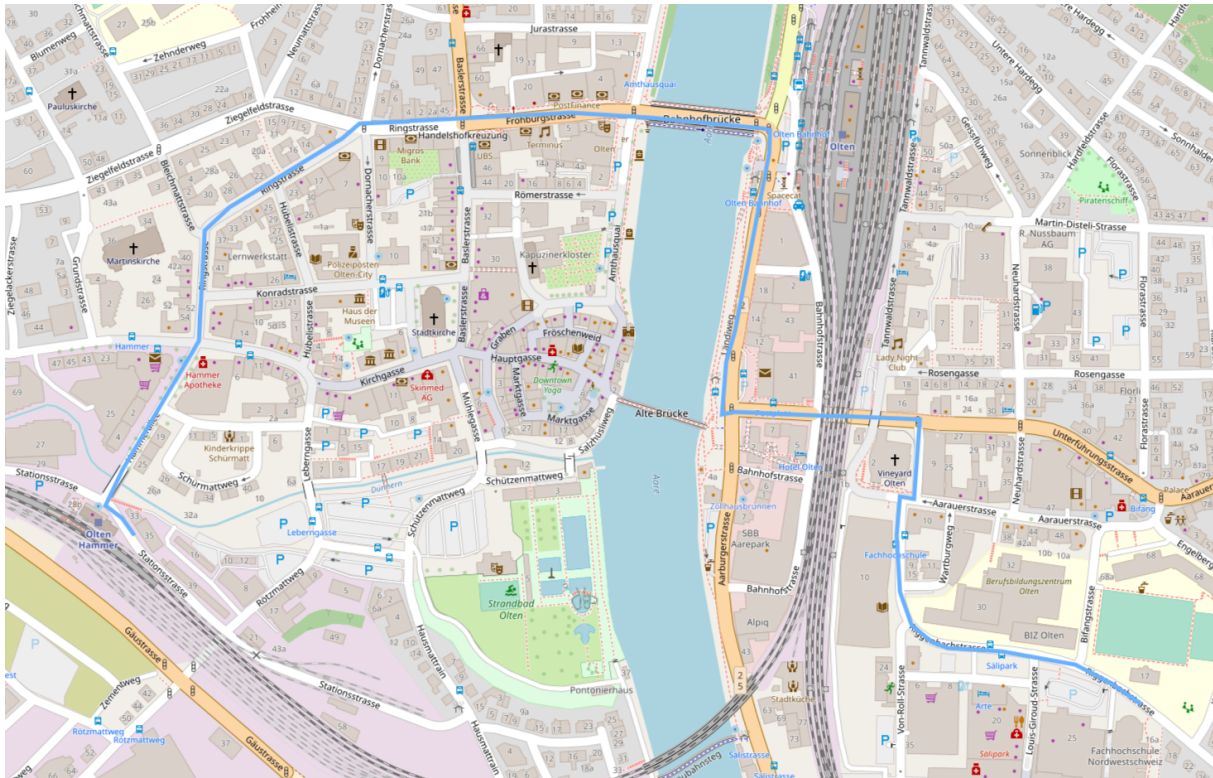
- Sample Java code for implementing Kafka Stream Processing applications is available on Moodle as a pre configured Maven Project.



- Please take a look at the provided “Utils.java” file as it contains several helper functions that make the overall implementation a lot easier. Of course you can also write your own implementations.
- It is probably the easiest to start the implementation from the provided sample project as it is already configured with all the needed dependencies.
- If you want to simplify the implementation, you can also implement both processing stages in the same application thus skipping the topic/channel that links the two applications together.

## 2 Sample Data

- For simplicity: All drivers take the same route:



**Figure 1:** Simulated Route

- The updates on the driver positions are available from an Apache Kafka topic (See Section “Kafka Web UI” if you want to browse the messages/events in the topic):
  - Kafka broker: 192.168.111.10:9092
  - Topic: driver-position
  - Message Format: (DeliveryID, Timestamp, Latitude, Longitude)
  - Example Message:

```
1 id: 6, time: 2024-04-01T12:01:21.123647949+02:00, lat: 47.35203, lon: 7.905917
```

```
m@x1c:~/Downloads/_chrome/kafka_2.13-3.5.1 — /usr/lib/jvm/java-17//...
id: 143, time: 2023-10-10 15:14:14.145717871 +02:00, lat: 47.34985, lon: 7.90599
id: 140, time: 2023-10-10 15:14:14.341260995 +02:00, lat: 47.34698, lon: 7.911312
id: 141, time: 2023-10-10 15:14:15.443127183 +02:00, lat: 47.34748, lon: 7.908689
id: 142, time: 2023-10-10 15:14:15.844295365 +02:00, lat: 47.349438, lon: 7.908521
id: 140, time: 2023-10-10 15:14:15.641260995 +02:00, lat: 47.346935, lon: 7.911396
id: 147, time: 2023-10-10 15:14:15.752274427 +02:00, lat: 47.35104, lon: 7.899227
id: 144, time: 2023-10-10 15:14:16.847214756 +02:00, lat: 47.352, lon: 7.906335
id: 148, time: 2023-10-10 15:14:17.853519739 +02:00, lat: 47.349224, lon: 7.89839
id: 143, time: 2023-10-10 15:14:17.645717871 +02:00, lat: 47.349792, lon: 7.905977
id: 141, time: 2023-10-10 15:14:17.743127183 +02:00, lat: 47.34743, lon: 7.90889
id: 144, time: 2023-10-10 15:14:18.947214756 +02:00, lat: 47.351994, lon: 7.906411
id: 145, time: 2023-10-10 15:14:19.649464015 +02:00, lat: 47.352066, lon: 7.902611
id: 144, time: 2023-10-10 15:14:20.047214756 +02:00, lat: 47.351986, lon: 7.906495
id: 141, time: 2023-10-10 15:14:19.943127183 +02:00, lat: 47.347424, lon: 7.909086
id: 144, time: 2023-10-10 15:14:21.047214756 +02:00, lat: 47.351974, lon: 7.906583
id: 145, time: 2023-10-10 15:14:21.649464015 +02:00, lat: 47.352066, lon: 7.902787
id: 141, time: 2023-10-10 15:14:21.643127183 +02:00, lat: 47.347416, lon: 7.909236
id: 145, time: 2023-10-10 15:14:22.549464015 +02:00, lat: 47.352066, lon: 7.90287
id: 147, time: 2023-10-10 15:14:21.852274427 +02:00, lat: 47.351093, lon: 7.899257
id: 147, time: 2023-10-10 15:14:22.352274427 +02:00, lat: 47.351112, lon: 7.899279
id: 146, time: 2023-10-10 15:14:23.350800020 +02:00, lat: 47.351864, lon: 7.900939
id: 148, time: 2023-10-10 15:14:23.553519739 +02:00, lat: 47.349503, lon: 7.898643
id: 142, time: 2023-10-10 15:14:23.844295365 +02:00, lat: 47.349438, lon: 7.908521
id: 142, time: 2023-10-10 15:14:24.744295365 +02:00, lat: 47.349365, lon: 7.908514
id: 147, time: 2023-10-10 15:14:24.452274427 +02:00, lat: 47.351196, lon: 7.899394
id: 141, time: 2023-10-10 15:14:24.843127183 +02:00, lat: 47.347404, lon: 7.909587
id: 144, time: 2023-10-10 15:14:26.047214756 +02:00, lat: 47.351974, lon: 7.906583
id: 145, time: 2023-10-10 15:14:26.749464015 +02:00, lat: 47.35209, lon: 7.903334
id: 144, time: 2023-10-10 15:14:27.147214756 +02:00, lat: 47.351902, lon: 7.90658
id: 146, time: 2023-10-10 15:14:27.950800020 +02:00, lat: 47.351902, lon: 7.901037
```

A REST Service can be used to determine if a delivery is delayed based on the current GPS position and time when the position was reported:

- URL:

`http://192.168.111.11:8080/route/ID?time=TIME&lat=LAT&lon=LON`

- With the UPPER case words as placeholders. Note: the time field must be URL encoded<sup>1</sup> and in the same format as provided in the driver position updates.
- Example URL (time field is already encoded): `http://192.168.111.11:8080/route/10?time=2023-10-17T16%3A00%3A56.558554461%2B02%3A00&lat=47.34876&lon=7.908264`

- Result: The service returns the number of seconds that the driver is delayed. Number can be negative if the driver is faster than expected.

<sup>1</sup>`String output = URLEncoder.encode("input string", StandardCharsets.UTF_8.toString())`



All Sample Services / Data Sources are available through the VPN only.

### 3 Step 1 - Definition of a processing flow for the given Scenario

Let's assume that we want to determine a significant delay if the delay is larger than 3 minutes. If we detect such a case, we want to be informed by reporting that state on a dashboard.

Based on those assumptions, define how the processing can take place based on the Complex Event Processing concept.

Note: please define the processing in two separate stages.

Expected Results:

- Definition of the processing flow
- Definition of the event Model
- Definition of the needed Topics

### 4 Step 2 - Obtaining Data Stream with Position Updates and Requesting the delay of the Delivery Driver

To process the position updates we use Kafka's stream processing functionalities to implement a small Java Program that contains the necessary logic.

The application needs to take the following steps:

1. Connect to the Apache Kafka broker
2. Register to the Topic "driver-position" to obtain the stream of update events
3. Request the delay of the driver towards the planned route by asking the mentioned REST Service.
4. Generate a new Event and publish it to the topic "group1234-route-timing".

### 5 Step 3 - Determine if the delay is significant

We once again use Kafka's Stream Processing functionalities to determine if a delay is significant enough (Delay larger than 3 minutes)

The approach is similar to the previous step:

1. Connect to the Apache Kafka broker

2. Register to the Topic “group1234-route-timing” to obtain the stream of route timing events
3. Check if the delay is greater than 3 minutes.
4. Print message to console so that we know that we found such a case.

## 6 Step 4 - Inform the customer / update the dashboard

A simple dashboard is available via <http://192.168.111.11:8080/status/>. It displays delay information as a simple table based on the information retrieved from the Kafka Topic “delays”<sup>2</sup>. Each message / event published to this topic needs to be in the following format:

```
1 id: 123, delay: 321
```

With ID as the Delivery ID and delay as the delay in seconds.

Only correctly formatted Events/Messages are consumed by the dashboard

**Actual Task:** Change the code from Step 2 so that we publish an Event in the correct format to the “delays” topic.

## 7 Utilities

### 7.1 Kafka Web UI

You can check which Topics are available on the Kafka broker and browse messages via the provided “Kouncil” Web UI:

- URL: <http://192.168.111.12:8080/topics>
- User: viewer
- Password: viewer
- **Please don't change any settings** as the instance is shared between all groups

---

<sup>2</sup>The basic dashboard is not distinguishing between the groups, so all groups publish to the same topic.